R.M.K
GROUP OF INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22CS602-OBJECT ORIENTED SOFTWARE ENGINEERING

DEPARTMENT : CSE/ADS/CSD

BATCH : 2023-2027

YEAR/SEMESTER : III/06

Prepared/Updated By :

Dr. A. Thilagavathy CSE/RMKEC	Dr. G. Manikandan, ADS/RMKEC
Mr. M. Vengateshwaran CSE/RMKEC	Mr. S. Bargunan, CSD/ RMKEC

DATE OF PREPARATION/ UPDATION: 02.02.2026

Table of Contents

S.NO	Topic	Page No.
1.	Course Objectives	5
2.	Pre-Requisites	6
3.	Syllabus	7
4.	Course outcomes	8
5.	CO- PO/PSO Mapping	9
6.	Unit -III Lecture Plan	11
7.	Activity based learning	12
8.	Lecture notes	13
9.	Assignments	68
10.	Part A Q&A (WITH K LEVEL AND CO)	69
11.	Part B Qs (WITH K LEVEL AND CO)	72
12.	Supportive online Certification courses	73
13.	Real time Applications in day to day life and to Industry	74
14.	Self-Learning/Enrichment Topics	75
15.	Assessment Schedule	79
16.	Prescribed Textbooks & Reference Books	80
17.	Mini Project / Activity	81

Course Objectives

The Course will enable learners to:

- Explain software engineering principles and activities involved in building large software programs.
- Describe the process of requirements gathering, analysis and unified modelling
- Illustrate the object oriented design process.
- Analyse various traditional and object oriented testing methods
- Apply estimation techniques, schedule project activities and compute pricing..

R.M.K GROUP OF INSTITUTIONS

Pre-Requisites

Semester VI

22CS602 - OBJECT ORIENTED SOFTWARE ENGINEERING

Semester II & III

20CS201-DATA STRUCTURES,
20CS302-OBJECT ORIENTED PROGRAMMING

R.M.K GROUP OF INSTITUTIONS

SYLLABUS

		L T P C
22CS602	**OBJECT ORIENTED SOFTWARE ENGINEERING(Lab Integrated)**	**2 0 2 3**

UNIT I PRODUCT AND PROCESS **6+6**

The Nature of Software – Defining the Discipline – The Software Process – Process models – Prescriptive Process Models – Product and Process – Agility and Process – What is an Agile Process?– Scrum– Other Agile Framework – Kanban – DevOps

UNIT II REQUIREMENTS AND UNIFIED MODELING **6+6**

Requirements Engineering – Establishing the Groundwork: Non-functional Requirements – Requirements Gathering – Developing Use Cases – Negotiating and Validating Requirements. Unified Modeling Language – Introduction – Static and Dynamic Models – Modelling – Introduction to the UML – UML Diagrams – UML Class Diagrams – Use-Case Diagram – UML Dynamic Modelling.

UNIT III OBJECT ORIENTED ANALYSIS AND DESIGN **6+6**

Object oriented Analysis process – Business object Analysis – Use-case driven OOA – Business process modelling – Use case model. Design Concepts -Design Process – Design Concepts – Design Model: Design Principles and Design Elements. Architectural Design– Designing class- based components - Conducting Component Level Design – User Interface Analysis and Design – Pattern-Based Software Design

UNIT IV SOFTWARE TESTING **6+6**

Component Level: A Strategic Approach to Software Testing – White-Box Testing – Black Box Testing- Object Oriented Testing. Integration Level: Integration Testing – AI and Regression Testing- – Integration Testing in the OO Context Specialized Testing for Mobility: Web application Testing – Web Testing Strategies – Security Testing – Performance Testing – Real time Testing – Testing AI Systems – Testing Virtual Environments

UNIT V SOFTWARE QUALITY AND PROJECT MANAGEMENT **6+6**

Software Metrics and Analytics: Software Measurement – Product Metrics. Creating a Viable Software Plan: The Project Planning Process – Software Scope and Feasibility – Decomposition and Estimation Techniques – Project Scheduling. Risk Management: Reactive Versus Proactive Risk Strategies – Risk Identification – Risk Projection – The RMMM Plan. Software Process Improvement: The SPI Process – The CMMI.

R.M.K GROUP OF INSTITUTIONS

Course outcomes

Upon completion of the course, the students will be able to:

Course Outcome s	Description	Level in Bloom's Taxonomy
CO1	Summarize software engineering principles and activities involved in building large software programs	K2
CO2	Describe the different phases of software development.	K2
CO3	Explain the basics of OOAD and develop software using object oriented design.	K2
CO4	Illustrate the different stages of the design process with a case study.	K3
CO5	Develop miniprojects using the application of object oriented analysis and design.	K3
CO6	Apply different testing strategies to develop efficient projects.	K3

CO- PO/PSO Mapping

Course OutComes	Level of CO	Program Outcomes												Program Specific Outcomes		
		K3	K4	K4	K5	K3, K5, K6	A3	A2	A3	A3	A3	A3	A2	PSO-1	PSO-2	PSO-3
		PO-1	PO-2	PO-3	PO-4	PO-5	PO-6	PO-7	PO-8	PO-9	PO-10	PO-11	PO-12			
CO1	K2	2	1	1	1	1	-	-	1	1	1	-	1	2	2	-
CO2	K2	2	1	1	1	1	-	-	1	1	1	-	1	2	2	-
CO3	K2	3	2	2	1	1	-	-	1	1	1	-	1	2	2	-
CO4	K3	3	2	2	1	1	-	-	1	1	1	-	1	2	2	-
C05	K3	3	2	2	1	1	-	-	1	1	1	-	1	2	2	-
C06	K3	3	2	2	1	1	-	-	1	1	1	-	1	2	2	-

R.M.K GROUP OF INSTITUTIONS

UNIT - III

OBJECT ORIENTED ANALYSIS AND DESIGN

SNO	TOPIC	NO OF PERIODS	PRETAINING CO	TAXONOMY LEVEL	MODE OF DELIVERY
1	Object oriented Analysis process	1	CO3	K2	
2	Business object Analysis – Use-case driven OOA	1	CO3	K2	
3	Business process modelling – Use case model	1	CO3	K2	
4	Design Concepts - Design Process	1	CO3	K2	
5	Design Principles and Design Elements. Architectural Design	1	CO3	K2	
6	Designing class- based	1	CO3	K2	
7	components - Conducting Component Level Design	1	CO3	K3	
8	User Interface Analysis and Design	1	CO3	K3	
9	Pattern-Based Software Design	1	CO3	K2	

Activity based learning

Exercise:

- Learn by solving problems – Tutorial Sessions can be conducted
- Learn by questioning - Conduction of quiz using online tools
- Learn by doing hands-on --Requirements document can be prepared

`

Lecture Notes

UNIT III OBJECT ORIENTED ANALYSIS AND DESIGN

Object oriented Analysis process – Business object Analysis – Use-case driven OOA – Business process modelling – Use case model. Design Concepts -Design Process – Design Concepts – Design Model: Design Principles and Design Elements. Architectural Design –Designing class- based components - Conducting Component Level Design – - User Interface Analysis and Design – Pattern-Based Software Design

R.M.K GROUP OF INSTITUTIONS

INTRODUCTION

The first step in finding an appropriate solution to a given problem is to understand the problem and its domain. The main objective of the analysis is to capture a complete, unambiguous, and consistent picture of the requirements of the system and what the system must do to satisfy the users' requirements and needs. This is accomplished by constructing several models of the system that concentrate on describing what the system does rather than how it does it, Separating the behavior of a system from the way that behavior is implemented requires viewing the system from the perspective of the user rather than that of the machine.

What is Analysis?

Analysis is the process of transforming a problem definition from a fuzzy set of facts and myths into a coherent statement of a system's requirements.

In Software process, you must analyze how the users will use the system and what is needed to accomplish the system's operational requirements. Analysis involves a great deal of interaction with the people who will be affected by the system, including the actual users and anyone else on whom its creation will have an impact.

The analyst has four major tools at his or her disposal for extracting information about a system:

1.Examination of existing system documentation 2.Interviews
3.Questionnaire
4.Observation

WHY ANALYSIS IS A DIFFICULT ACTIVITY?

Analysis is a creative activity that involves understanding the problem,its constraints, and methods of overcoming those constraints: This is an iterative process that goes on until the problem is well understood.

R.M.K GROUP OF INSTITUTIONS

The three most common sources of requirement difficulties:

1.Fuzzy descriptions
2.Incomplete requirements
3.Unnecessary features

Fuzzy descriptions

A common problem that leads to requirement ambiguity is a fuzzy and am- biguous description, such as "fast response time" or "very easy and very secure updating mechanisms." A requirement such as fast response time is open to interpretation, which might lead to user dissatisfaction if the user's interpretation of a fast response is different from the systems analyst's interpretation .

Incomplete requirements

Incomplete requirements mean that certain requirements necessary for success- ful system development are not included for a variety of reasons. These reasons could include the users' forgetting to identify them. high cost, politics within the business, or oversight by the system developer.

Unnecessary features

When addressing features of the system, keep in mind that every additional fea- ture could affect the performance, complexity, stability, maintenance, and support costs of an application. For example. if the primary task is .selecting a single object, extending it to .support selection of multiple objects could make the frequent, simple task more difficult to carry out.

A number of other factors also may affect the design of an application. For example, deadlines may require delivering a product to market .with a minimal design process, or comparative evaluations may force considering additional features. Remember that additional features and shortcuts can affect the product.

R.M.K GROUP OF INSTITUTIONS

BUSINESS OBJECT ANALYSIS: UNDERSTANDING THE BUSINESS LAYER

Business object analysis is a process of understanding the system's requirements and establishing the goals of an application. The main intent of this activity is to understand users' requirements.

Outcome:

The outcome of the business object analysis is to identify classes that make up the business layer and the relationships that play a role in achieving system goals.To understand the users' requirements, we new to find out how they "use" the system.This can be accomplish by developing use cases. Use cases are scenarios for understanding system requirements.

USE-CASE DRIVEN OBJECT-ORIENTED ANALYSIS: THE UNIFIED APPROACH

The object-oriented analysis (00A) phase of the unified approach uses actors and use cases to describe the system from the users' perspective. The actors are external factors that interact with the system; use cases are scenarios that describe how actors use the system. The use cases identified here will be involved throughout the development process

The 00A process consists of the following steps.

1. Identify the actors:
   - Who is using the system?
   - Or, in the case of a new system, who will be using the system?
2. Develop a simple businessprocessmodel using UNIL activitydiagram.
3. Develop the use case:
   - What are the users doing with the system?
   - Or. in case of the new system, what will users be doing with the system?
   - Use cases provide us with comprehensive documentation of the system under study.
4. Prepare interaction diagrams:
   - Determine the sequence.
   - Develop collaboration diagrams.
5. Classification—develop a static MIL class diagram:
   - identify classes.
   - Identify relationships.
   - Identify attributes.
   - Identify methods.
6. Iterate and refine: If needed. repeat the preceding steps.

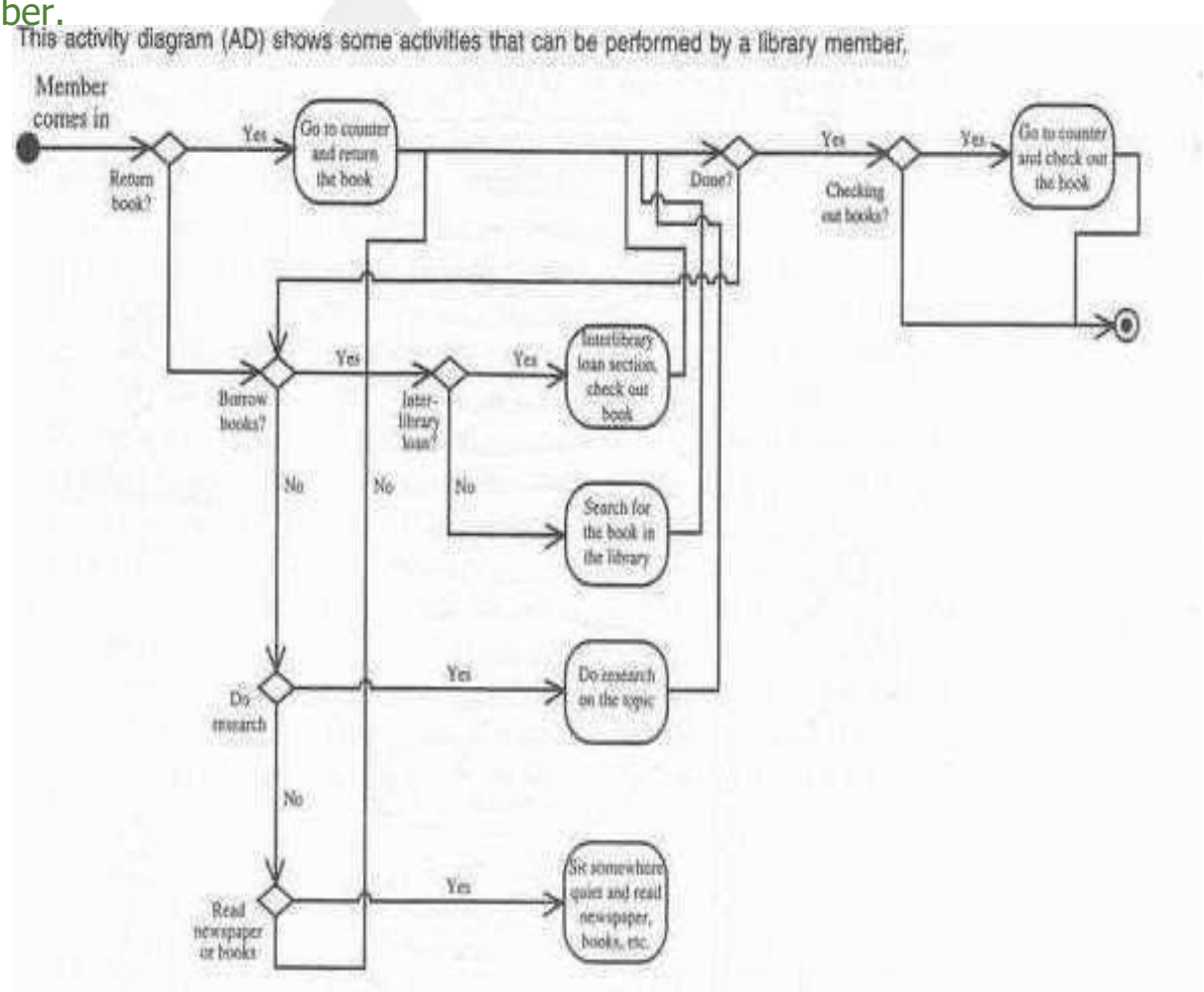
R.M.K GROUP OF INSTITUTIONS

BUSINESS PROCESS MODELING

This is not necessarilythe startof every project.but when required. businessprocessesand user requirements may be modeled and recorded to any level of detail.This may include modeling as-is processes and the applications that support them and any number of phased. would-be models of reengineered processes or implementation of the system.
Theseactivitieswould be enhanced and supported by using an activitydiagram.

Businessprocessmodelingcan be verytime consuming. so the mainidea should be to get a basic model without spending too much time on the process.

Merits:
The advantage of developing a business process model is that it makes you more familiar with the system and therefore the user requirements and alsoaids in developing use cases. For example. let us definethe steps or activitiesinvolved in using yourschool library. These activitiescan be represented with an activity diagram Developing an activity diagram of the business process can give us a better understanding of what sort of activities are performed in a library by a library member.



USE CASE MODEL.

Use cases are scenarios for understanding system requirements. A use-case model can be. instrumental in project development. planning, and documentation of systems requirements. A use case is an interaction between users and a system: it captures the goal of the users and the responsibility of the system to its users.

Example:

Take a car, typical uses of a car include "take you different places" or "haul your stuff' or a. user may want to use it "off the road," The rise-case model de-scribes the uses of the system and shows the courses of events that can be per-formed. In other words, it shows a system in terms of its users and how it is be- ing used from a user point of view, Furthermore, it defines what happens in the system when the use case is performed. in essence, the use-case model tries to sys- tematically identify uses of the system and therefore. the system's responsibilities.A use-case model also candiscover classesand the relationships'amongsubsys-tems of the systems.

A use-case model is a model of how different types of users interact with the system to solve a problem. As such, it describes the goals of the users, the interactions between the users and the system, and the required behavior of the system in satisfying thesegoals.

A use-casemodelconsistsof a number of modelelements.

The mostimportant modelelements are:

use cases,
actors and
the
relationshipsbetween them.

A use-case diagram is used to graphically depict a subset of the model to simplify communications. There will typically be several use-case diagrams associated with a given model, each showing a subset of the model elements relevant for a particular purpose. The same model element may be shown on several use-case diagrams, but each instance must be consistent. If tools are used to maintain the use-case model, this consistency constraint is automated so that any changes to the model element (changing the nameforexample) will be automatically reflected on every use-casediagram thatshows thatelement.

R.M.K GROUP OF INSTITUTIONS

The use-case model may contain packages that are used to structure the model to simplify analysis, communications, navigation, development, maintenance and planning.

Much of the use-case model is in fact textual, with the text captured in the Use-Case Specifications that are associated with each use-case model element. These specifications describe the flow of events of the use case.
The use-case model serves as a unifying thread throughout system development. It is used as the primary specification of the functional requirements for the system, as the basis for analysis and design, as an input to iteration planning, as the basis of defining test cases and as the basis for user documentation

USE CASE MODEL.

Use cases are scenarios for understanding system requirements. A use-case model can be. instrumental in project development. planning, and documentation of systems requirements. A use case is an interaction between users and a system: it captures the goal of the users and the responsibility of the system to its users.
Basic model elements:

The use-case model contains as a minimum, the following basic model elements.
Actor

A model element representing eachactor. Properties include the actors name and brief description.

Use Case
A model element representing each use case. Properties include the use case name and use case specification.

Associations
Associations are used to describe the relationships between actors and the use cases they participate in. This relationship is commonly known as a "communicates-association".

Use-Case Package

A model element used to structurethe use casemodel to simplifyanalysis, communications, navigation, and planning. If there are many use cases or actors, you can use use-case packages to further structure the use-casemodel in much the samemanner youuse foldersor directories to structurethe information on your hard-disk.

R.M.K GROUP OF INSTITUTIONS

Generalizations in Use case:

A relationship between actors to support re-use of common properties.
Dependencies

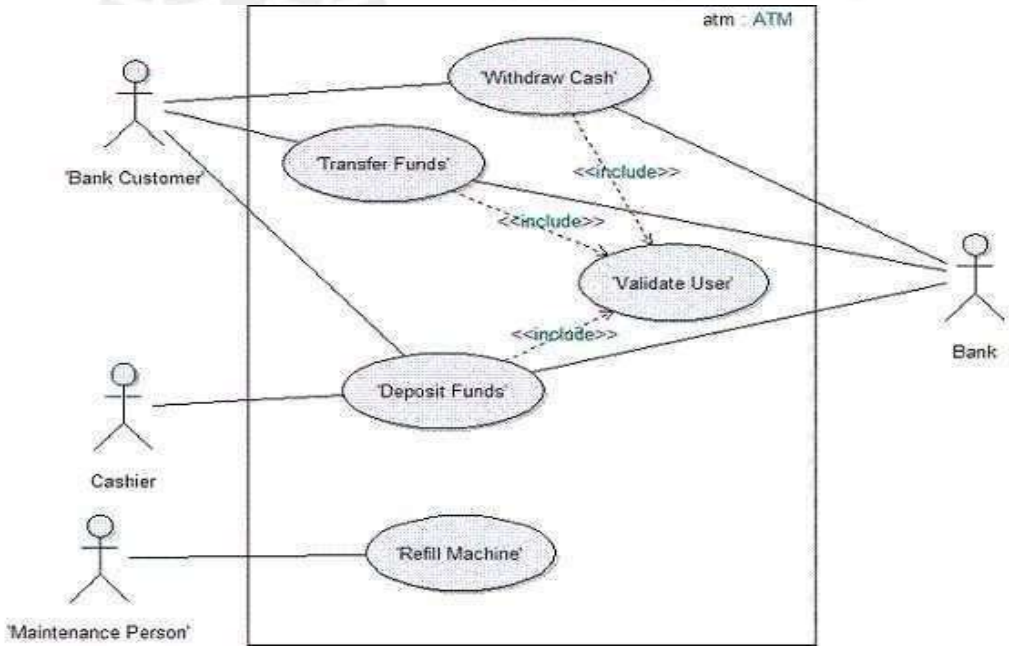
A number of dependency types between use cases are defined in UML. In particular,
<<extend>> and <<include>>.

<<extend>> is used to include optional behavior from an extending use case in an extended use case.

<<include>> is used to include common behavior from an included use case into a base use case in order to support re-use of common behavior.

The latter is the most widely used dependency and is useful for:

- Factoring out behavior from the base use case that is not necessary for the understanding of the primary purpose of the use case to simplify communications.
- Factoring out behavior that is in common for two or more use cases to maximize re-use, simplify maintenance and ensure consistency.



R.M.K GROUP OF INSTITUTIONS

ATM Use-Case Diagram

This diagram shows the subject (atm:ATM), four actors (Bank Customer, Bank, Cahier and Maintenance Person), five use cases (Withdraw Cash, Transfer Funds, Deposit Funds, Refill Machine and Validate User), three <<includes>> dependencies, and the associations between the performing actors and the use cases.

The use cases Withdraw Cash, Deposit Funds, and Transfer Funds all need to include how the customer is identified to the system. This behavior can be extracted to a new inclusion use case called Validate User, which the three base use cases <<include>>. The base use cases are independent of the method used for identification, and it is therefore encapsulated in the inclusion use case. From the perspective of the base use cases, it does not matter whether the method for identification is to read a magnetic bank card, or perform a retinal scan. They only depend on the result of Validate Customer.

R.M.K GROUP OF INSTITUTIONS

Design process

Design creates a representation or model of the software, the design model provides detail about software architecture, data structures, interfaces, and components that are necessary to implement the system.

Software design sits at the technical kernel of software engineering and is applied regardless of the software process model that is used. Beginning once software requirements have been analyzed and specified, **software design is the first** of three technical activities—design, code generation, and test—that are required to build and verify the software. Each activity transforms information in a manner that ultimately results in validated computer software.

From Analysis Model to Design Model

Each element of the analysis model provides information that is necessary to create the four design models

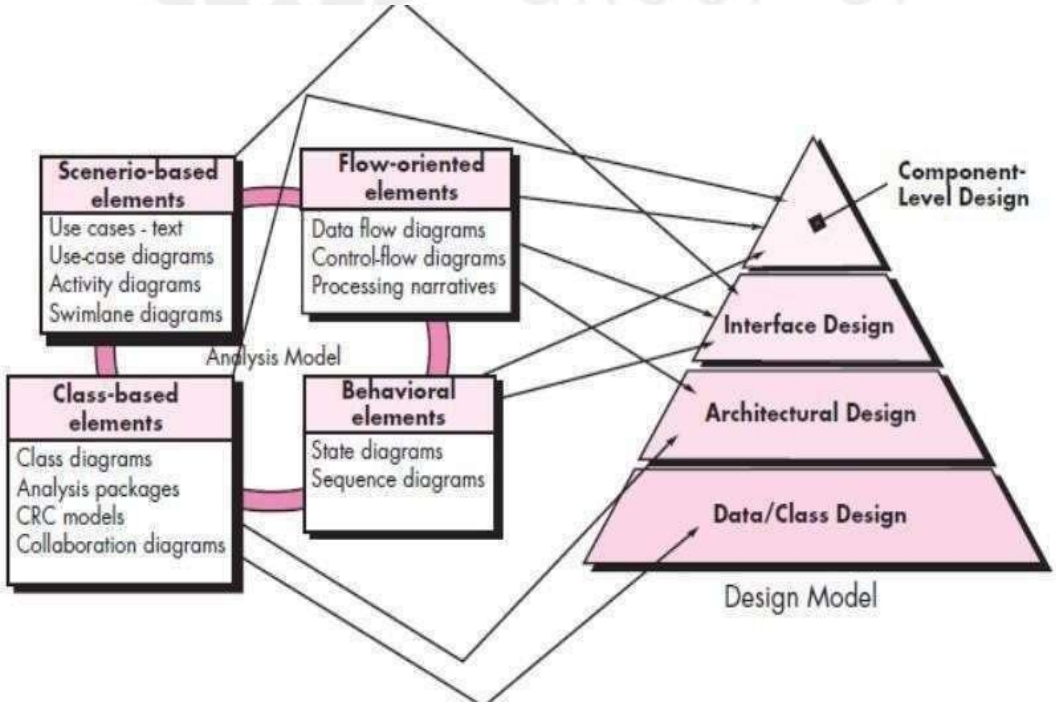


Fig : Translating from Analysis Model to Design Model

- The data/class design transforms analysis classes into design classes along with the data structures required to implement the software
- The architectural design defines the relationship between major structural elements of the software; architectural styles and design patterns help achieve the requirements defined for the system
- The interface design describes how the software communicates with systems that interoperate with it and with humans that use it
- The component-level design transforms structural elements of the software architecture into a procedural description of software components

Design Quality Guidelines

- A design should exhibit an architecture that a) Has been created using recognizable architectural styles or patterns b) Is composed of components that exhibit good design characteristics c) Can be implemented in an evolutionary fashion, thereby facilitating implementation and testing
- A design should be modular; that is, the software should be logically partitioned into elements or subsystems
- A design should contain distinct representations of data, architecture, interfaces, and components
- A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns
- A design should lead to components that exhibit independent functional characteristics
- A design should lead to interfaces that reduce the complexity of connections between components and with the external environment
- A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis
- A design should be represented using a notation that effectively communicates its meaning

Design Concepts

- Abstraction—data, procedure, control
- Software Architecture—the overall structure of the software
- Patterns—"conveys the essence" of a proven design solution
- Separation of concerns—any complex problem can be more easily handled if it is subdivided into pieces
- Modularity—compartmentalization of data and function
- Information Hiding—controlled interfaces

R.M.K GROUP OF INSTITUTIONS

- Functional independence—single-minded function and low coupling
- Refinement—elaboration of detail for all abstractions
- Aspects—a mechanism for understanding how global requirements affect design
- Refactoring—a reorganization technique that simplifies the design

Abstraction:

- When you consider a modular solution to any problem, many levels of abstraction can be posed. At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment. At lower levels of abstraction, a more detailed description of the solution is provided.
- Procedural abstraction – a sequence of instructions that have a specific and limited function
- Data abstraction – a named collection of data that describes a data object

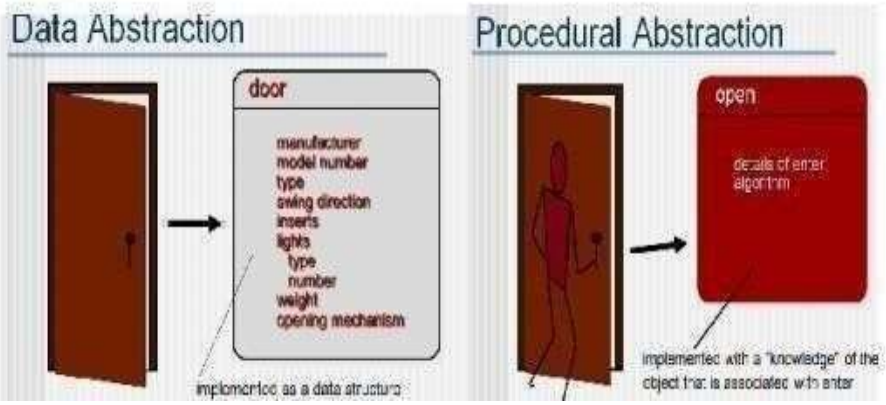


Fig : Abstraction

Reference Video:

Software Design
https://www.youtube.com/watch?v=F440S-HibGQ

Software Architecture

- "The overall structure of the software and the ways in which that structure provides conceptual integrity for a system." [SHA95a] Structural properties.

  This aspect of the architectural design representation defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another.

- For example, objects are packaged to encapsulate both data and the processing that manipulates the data and interact via the invocation of methods Extra-functional properties.

- The architectural design description should address how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.

- Families of related systems. The architectural design should draw upon repeatable patterns that are commonly encountered in the design of families of similar systems. In essence, the design should have the ability to reuse architectural building blocks.

- Given the specification of these properties, the architectural design can be represented using one or more of a number of different models [Gar95]. Structural models represent architecture as an organized collection of program components.

- Framework models increase the level of design abstraction by attempting to identify repeatable architectural design frameworks that are encountered in similar types of applications.

- Dynamic models address the behavioral aspects of the program architecture, indicating how the structure or system configuration may change as a function of external events.

- Process models focus on the design of the business or technical process that the system must accommodate. Functional models can be used to represent the functional hierarchy of a system.
- A number of different architectural description languages (ADLs) have been developed to represent these models

Patterns

- A design structure that solves a particular design problem within a specific context
- It provides a description that enables a designer to determine whether the pattern is applicable, whether the pattern can be reused, and whether the pattern can serve as a guide for developing similar patterns
- The intent of each design pattern is to provide a description that enables a designer to determine (1) whether the pattern is applicable to the current work,

  (2) whether the pattern can be reused (hence, saving design time) (3) whether the pattern can serve as a guide for developing a similar, but functionally or structurally different pattern

Separation of Concerns

- Any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently.
- A concern is a feature or behavior that is specified as part of the requirements model for the software.
- By separating concerns into smaller, and therefore more manageable pieces, a problem takes less effort and time to solve.

Modularity

- Separately named and addressable components (i.e., modules) that are integrated to satisfy requirements (divide and conquer principle) "modularity is the single attribute of software that allows a program to be intellectually manageable" [Mye78].
- Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer. In almost all instances, you should break the design into many modules, hoping to make understanding easier and as a consequence, reduce the cost required to build the software

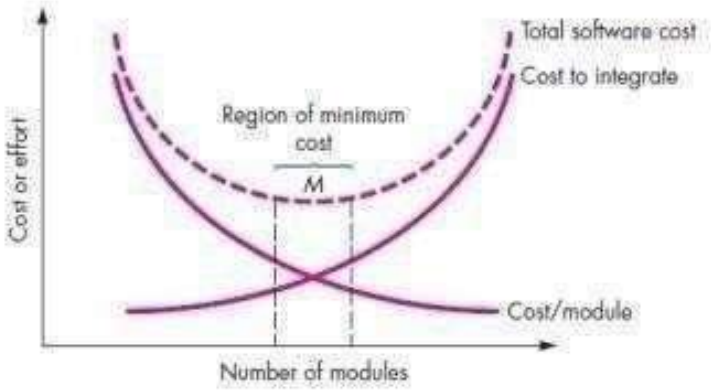


Fig: Modularity and software cost

the effort (cost) to develop an individual software module does decrease as the total number of modules increases.

Given the same set of requirements, more modules means smaller individual size. However, as the number of modules grows, the effort (cost) associated with integrating the modules also grows.

These characteristics lead to a total cost or effort curve shown in the figure.
There is a number, M, of modules that would result in minimum development cost, but we do not have the necessary sophistication to predict M with assurance

- The curves shown in Figure do provide useful qualitative guidance when modularity is considered.
- You should modularize, but care should be taken to stay in the vicinity of
- M. Under modularity or over modularity should be avoided
- You modularize a design so that development can be more easily planned; software increments can be defined and delivered; changes can be more easily accommodated; testing and debugging can be conducted more efficiently, and long-term maintenance can be conducted without serious side effects.

Information hiding

- The designing of modules so that the algorithms and local data contained within them are inaccessible to other modules.
- This enforces access constraints to both procedural detail and local data structures. The principle of information hiding suggests that modules be "characterized by design decisions that (each) hides from all others." In other words, modules should be specified and designed so that information (algorithms and data) contained within a module is inaccessible to other modules that have no need for such information

Functional independence

- Functional independence is achieved by developing modules with "single-minded" function and an "aversion" to excessive interaction with other modules.
- Cohesion is an indication of the relative functional strength of a module. A cohesive module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should (ideally) do just one thing. Coupling is an indication of the relative interdependence among modules. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.

Cohesion

- Cohesion is a natural extension of the information hiding concept.
- A cohesive module performs a single task within a software procedure, requiring little interaction with procedures being performed in other parts of a program.
- Stated simply, a cohesive module should (ideally) do just one thing.
- Cohesion may be represented as a "spectrum." Always strive for high cohesion, although the mid-range of the spectrum is often acceptable.
- The scale for cohesion is nonlinear. That is, low-end cohesiveness is much "worse" than middle range, which is nearly as "good" as high-end cohesion.
- At the low (undesirable) end of the spectrum, encounter a module that performs a set of tasks that relate to each other loosely, if at all. Such modules are termed **coincidentally cohesive**.
- A module that performs tasks that are related logically (e.g., a module that produces all output regardless of type) is **logically cohesive**.
- When a module contains tasks that are related by the fact that all must be executed with the same span of time, the module exhibits **temporal cohesion**.
- Combining the functions into a single module can serve only to increase the likelihood of error propagation when a modification is made to one of its processing tasks.
- Moderate levels of cohesion are relatively close to one another in the degree of module independence. When processing elements of a module are related and must be executed in a specific order, **procedural cohesion** exists.
- When all processing elements concentrate on one area of a data structure, **communicational cohesion** is present.
- High cohesion is characterized by a module that performs one distinct procedural task. As we have already noted, it is unnecessary to determine the precise level of cohesion.
- Rather it is important to strive for high cohesion and recognize low cohesion so that software design can be modified to achieve greater functional independence.

Coupling

- Coupling is a measure of interconnection among modules in a software structure. Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface

- Modules a and d are subordinate to different modules. Each is unrelated and therefore no direct coupling occurs. Module c is subordinate to module a and is accessed via a conventional argument list, through which data are passed.

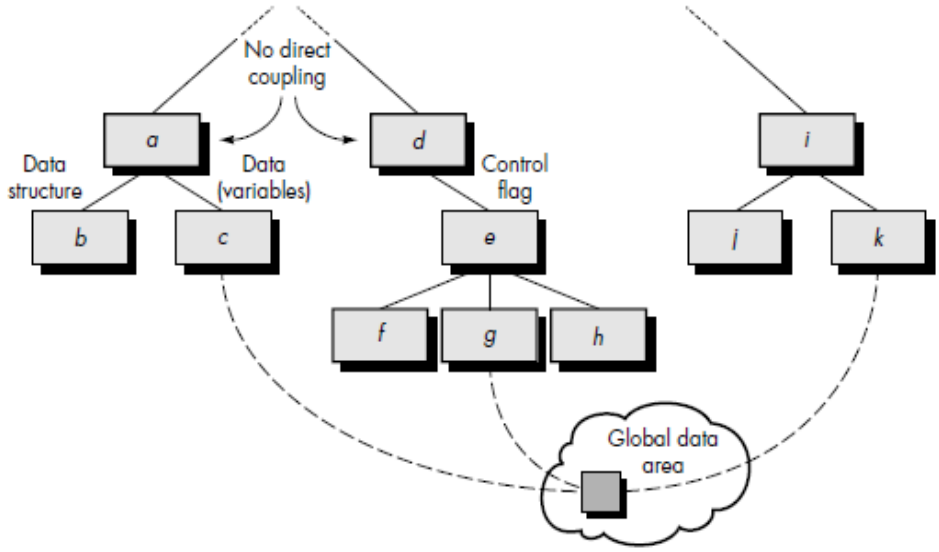


Fig : Types of Coupling

- As long as a simple argument list is present (i.e., simple data are passed; a one-to-one correspondence of items exists), **low coupling** (called **data coupling**) is exhibited in this portion of structure.

- A variation of data coupling, called **stamp coupling, is** found when a portion of a data structure (rather than simple arguments) is passed via a module interface. This occurs between modules b and a.

- At moderate levels, coupling is characterized by passage of control between modules.

- **Control coupling** is very common in most software designs and is shown in Figure, where a "control flag" (a variable that controls decisions in a subordinate or superordinate module) is passed between modules d and e. Relatively high levels of coupling occur when modules are tied to an environment external to software. For example, I/O couples a module to specific devices, formats, and communication protocols.

- **External coupling** is essential, but should be limited to a small number of modules with a structure. High coupling also occurs when a number of modules reference a global data area. **Common coupling**, as this mode is called, is shown in Figure. Modules c, g, and k each access a data item in a global data area (e.g., a disk file or a globally accessible memory area). Module c initializes the item.

- The highest degree of coupling, **content coupling**, occurs when one module makes use of data or control information maintained within the boundary of another module. Secondarily, content coupling occurs when branches are made into the middle of a module.

- This mode of coupling can and should be avoided.
- The coupling modes just discussed occur because of design decisions made when structure was developed.

- High cohesion – a module performs only a single task
- Low coupling – a module has the lowest amount of connection needed with other modules

Stepwise refinement

- Stepwise refinement is a top-down design strategy. A program is developed by successively refining levels of procedural detail. A hierarchy is developed by decomposing a macroscopic statement of function (a procedural abstraction) in a stepwise fashion until programming language statements are reached. Refinement is actually a process of elaboration. You begin with a statement of function that is defined at a high level of abstraction. That is, the statement describes function or information conceptually but provides no information about the internal workings of the function or the internal structure of the information. The original statement, providing more and more detail as each successive refinement (elaboration) occurs

Aspects

- An aspect is a representation of a cross-cutting concern. Consider two requirements, A and B. Requirement A crosscuts requirement B "if a software decomposition [refinement] has been chosen in which B cannot be satisfied without taking A into account

- It is important to identify aspects so that the design can properly accommodate them as refinement and modularization occur. In an ideal context, an aspect is implemented as a separate module (component) rather than as software fragments that are "scattered" or "tangled" throughout many components [Ban06]. To accomplish this, the design architecture should support a mechanism for defining an aspect—a module that enables the concern to be implemented across all other concerns that it crosscuts.

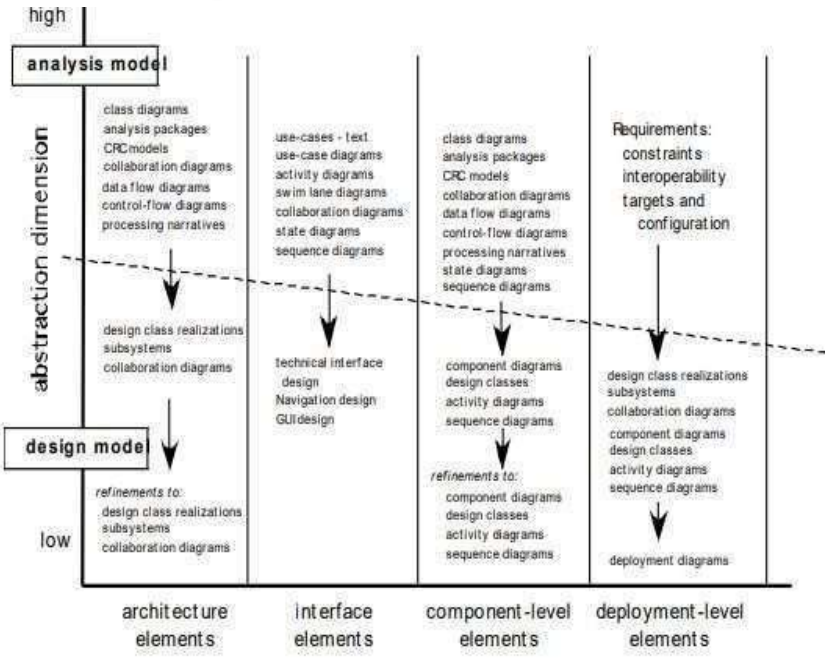
Refactoring

- "Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure."
- When software is refactored, the existing design is examined for
  - redundancy
  - unused design elements
  - inefficient or unnecessary algorithms
  - poorly constructed or inappropriate data structures
  - any other design failure that can be corrected to yield a better design.

The Design Model

- The design model can be viewed in two different dimensions
- (Horizontally)- The process dimension indicates the evolution of the parts of the design model as each design task is executed
- (Vertically) - The abstraction dimension represents the level of detail as each element of the analysis model is transformed into the design model and then iteratively refined
- Elements of the design model use many of the same UML diagrams used in the analysis model
- The diagrams are refined and elaborated as part of the design
- More implementation-specific detail is provided
- Emphasis is placed on
- Architectural structure and style
- Interfaces between components and the outside world
- Components that reside within the architecture

- **Data/class design elements**
- Creates a model of data and objects that is represented at a high level of abstraction
- This data model is then refined into progressively more implementation-specific representations that can be processed by the computer-based system.
- In many software applications, the architecture of the data will have a profound influence on the architecture of the software that must process it.
- The structure of data has always been an important part of software design.
- At the program component level, the design of data structures and the associated algorithms required to manipulate them is essential to the creation of high-quality applications.
- At the application level, the translation of a data model (derived as part of requirements engineering) into a database is pivotal to achieving the business objectives of a system.
- At the business level, the collection of information stored in disparate databases and reorganized into a "data warehouse" enables data mining



Architectural design

- Depicts the overall layout of the software.
- The architectural model [Sha96] is derived from three sources:
- Information about the application domain for the software to be built;
- specific requirements model elements such as data flow diagrams or analysis
- classes, their relationships and collaborations for the problem at hand; and
- The availability of architectural styles and patterns Interface design

Interface Design Elements

- Tells how information flows into and out of the system and how it is communicated among the components defined as part of the architecture. There are three important elements of interface design:
- the user interface (UI)
- external interfaces to other systems, devices, networks or other producers or consumers of information
- internal interfaces between various design components.
- Component-level design elements
- Describes the internal detail of each software component by way of data structure
- definitions, algorithms, and interface specifications

Deployment-level design elements

- Indicates how software functionality and subsystems will be allocated within the physical computing environment that will support the software

Architectural Design

- **Definition** :
- The software architecture of a program or computing system is the structure or structures of the system which comprise
- The software components
- The externally visible properties of those components
- The relationships among the components
- Architectural Design Process
- **Basic Steps**
- Creation of the data design
- Derivation of one or more representations of the architectural structure of the system
- Analysis of alternative architectural styles to choose the one best suited to customer requirements and quality attributes
- Elaboration of the architecture based on the selected architectural style
- A system architect selects an appropriate architectural style derived during system engineering and software requirements analysis

Architectural Styles

- Each style describes a system category that encompasses: (1) a set of components (e.g., a database, computational modules) that perform a function required by a system, (2) a set of connectors that enable "communication, coordination and cooperation" among components, (3) constraints that define how components can be integrated to form the system, and (4) semantic models that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

- Data-centered architectures
- Data flow architectures
- Call and return architectures
- Object-oriented architectures
- Layered architectures

Data-centered architectures

- A data store (e.g., a file or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store. Client software accesses a central repository.

- In some cases the data repository is passive.
- That is, client software accesses the data independent of any changes to the data or the actions of other client software. A variation on this approach transforms the repository into a "blackboard that sends notifications to client software when data of interest to the client changes.

- Data centered architectures promote integrability.

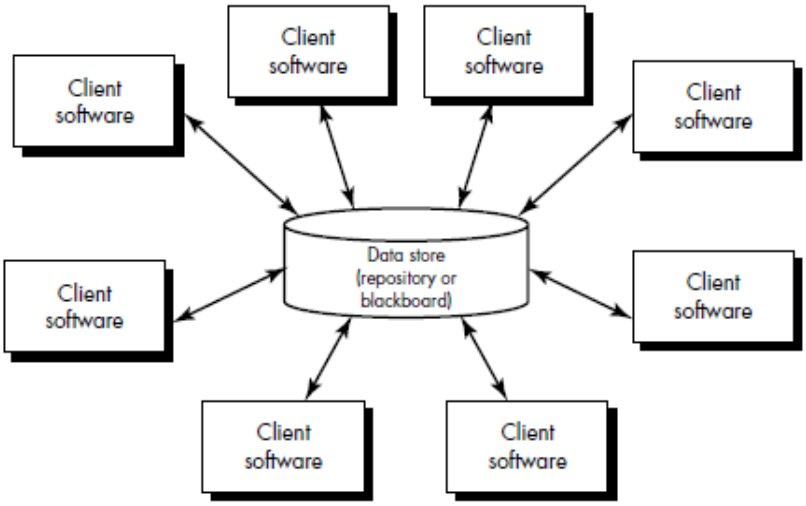


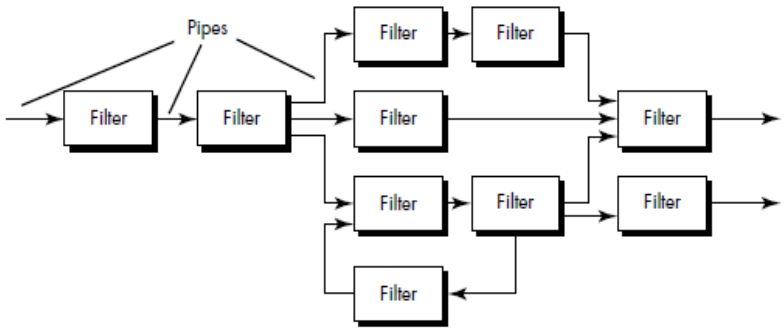
Fig: Data Centered Architecture

- In addition, data can be passed among clients using the blackboard mechanism (i.e., the blackboard component serves to coordinate the transfer of information between clients).

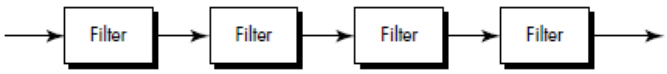
- Client components independently execute processes

- Data Flow Architecture

- This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data. A pipe-and-filter pattern has a set of components, called filters, connected by pipes that transmit data from one component to the next. Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output (to the next filter) of a specified form. However, the filter does not require knowledge of the workings of its neighboring filters. If the data flow degenerates into a single line of transforms, it is termed batch sequential. This structure accepts a batch of data and then applies a series of sequential components(filters) to transformit.



(a) Pipes and filters



(b) Batch sequential

Fig Data flow Architecture

- This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data.
- A pipe-and-filter pattern has a set of components, called filters, connected by pipes that transmit data from one component to the next.
- Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output (to the next filter) of a specified form.
- However, the filter does not require knowledge of the workings of its neighboring filters. If the data flow degenerates into a single line of transforms, it is termed batch sequential.
- This structure accepts a batch of data and then applies a series of sequential components (filters) to transform it.

Call and Return Architecture

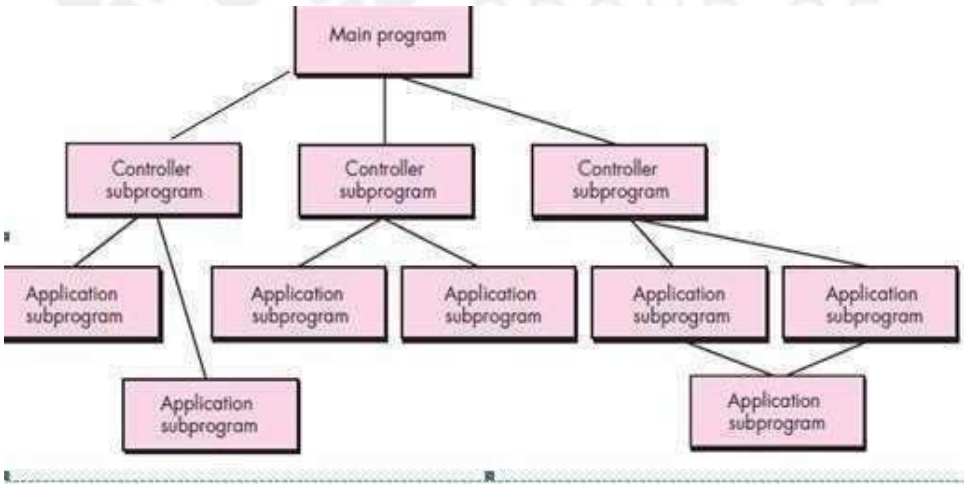


Fig: Call & Return Architecture

Reference Video:

Architectural Styles
https://www.youtube.com/watch?v=JLbo9Lvvy5M

- This architectural style enables you to achieve a program structure that is relatively easy to modify and scale.

- A number of sub styles exist within this category:
- **Main program/subprogram architectures**. This classic program structure invokes a number of program components that in turn may invoke still other components.
- **Remote procedure call architectures**. The components of a main program/subprogram architecture are distributed across multiple computers on a network.

Object-oriented architectures

- The components of a system encapsulate data and the operations that must be applied to manipulate the data.
- Communication and coordination between components are accomplished via message passing.

Layered architectures

- A number of different layers are defined, each accomplishing operations that progressively become closer to the machine instruction set

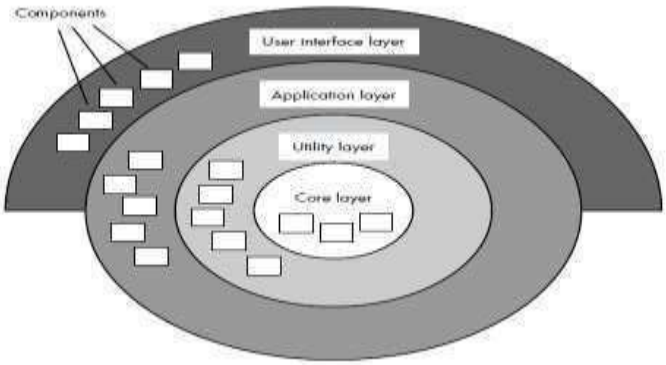


Fig: Layered architectures

- At the outer layer, components service user interface operations.
- At the inner layer, components perform operating system interfacing.
- Intermediate layers provide utility services and application software functions.

Architectural Design Process

- **Architectural Design Steps**
- Represent the system in context
- Define archetypes
- Refine the architecture into components
- Describe instantiations of the system

Represent the System in Context

- Use an architectural context diagram (ACD) that shows The identification and flow of all information into and out of a system The specification of all interfaces
- Any relevant support processing from/by other systems An ACD models the manner in which software interacts with entities external to its boundaries An ACD identifies systems that interoperate with the target system

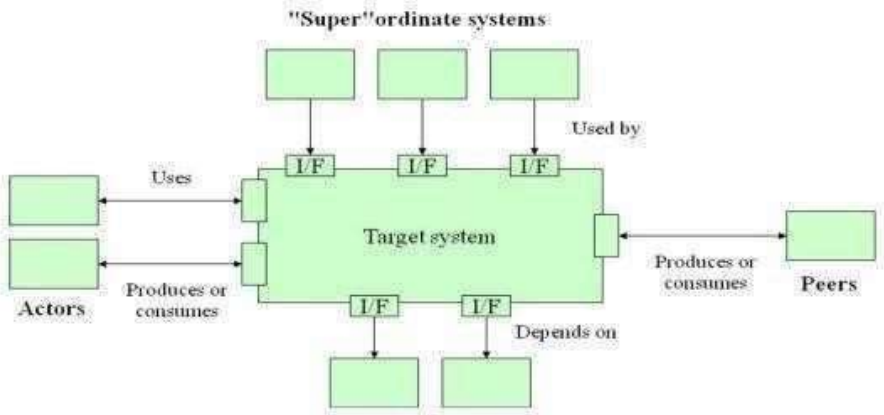
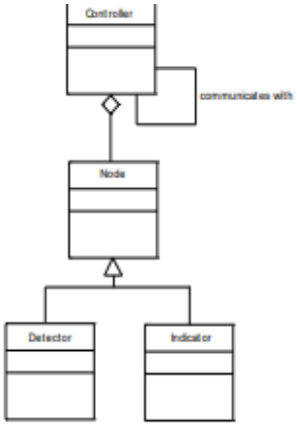


Fig: architectural context diagram

- Super-ordinate systems Use target system as part of some higher level processing scheme
- Sub-ordinate systems Used by target system and provide necessary data or processing
- Peer-level systems Interact on a peer-to-peer basis with target system to produce or consume data
- Actors People or devices that interact with target system to produce or consume data

Define Archetypes

- Archetypes indicate the important abstractions within the problem domain (i.e., they model information)
- It is also an abstraction from a class of programs with a common structure and includes class-specific design strategies and a collection of example program designs and implementations
- Only a relatively small set of archetypes is required in order to design even relatively complex systems
- The target system architecture is composed of these archetypes
- They represent stable elements of the architecture
- They may be instantiated in different ways based on the behavior of the system
- They can be derived from the analysis class model



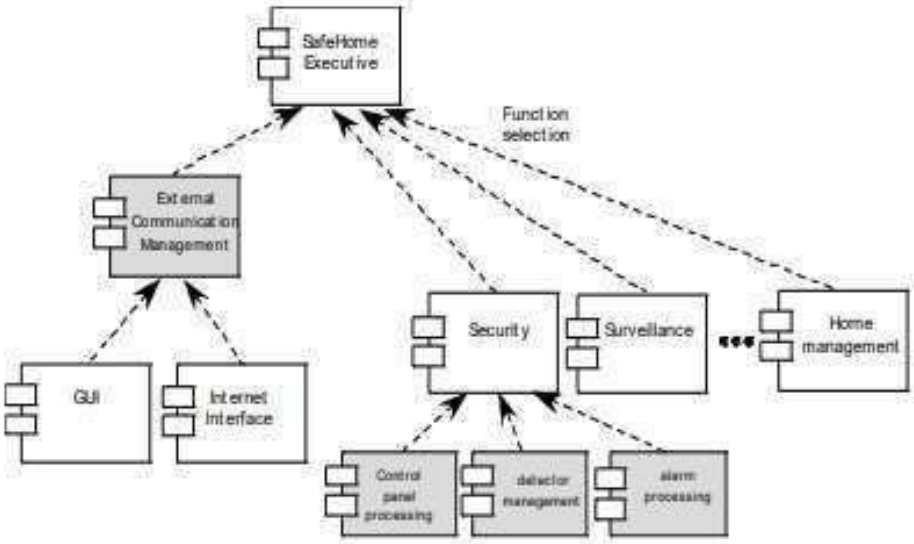
Refine the Architecture into Components

- Based on the archetypes, the architectural designer refines the software architecture into components to illustrate the overall structure and architectural style of the system

- These components are derived from various sources
- The application domain provides application components, which are the domain classes in the analysis model that represent entities in the real world

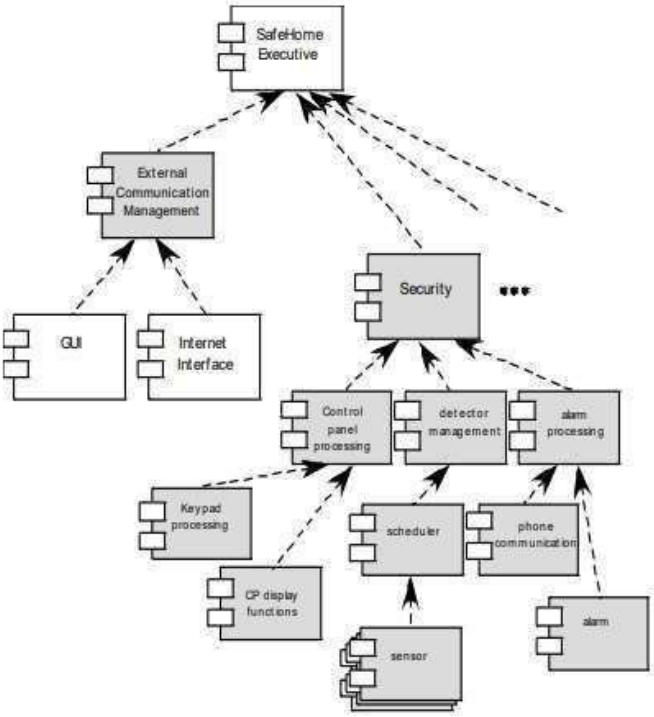
- The infrastructure domain provides design components (i.e., design classes) that enable application components but have no business connection
- Examples: memory management, communication, database, and task management

- The interfaces in the ACD imply one or more specialized components that process the data that flow across the interface



Describe Instantiations of the System

- An actual instantiation of the architecture is developed by applying it to a specific problem.
- This demonstrates that the architectural structure, style and components are appropriate.
- A UML component diagram can be used to represent this instantiation



Component-level design

- Component-level design defines the data structures, algorithms, interface characteristics, and communication mechanisms allocated to each software component.

- A software component is a modular building block for computer software
- It is a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces
- A component communicates and collaborates with
- Other components
- Entities outside the boundaries of the system
- Three different views of a component
  - An object-oriented view
  - A conventional view
  - A process-related view

Designing Class-Based Components

- **Component-level Design Principles**
- **Open-closed principle**
- A module or component should be open for extension but closed for modification
- The designer should specify the component in a way that allows it to be extended without the need to make internal code or design modifications to the existing parts of the component

- **Liskov substitution principle**
- Subclasses should be substitutable for their base classes
- A component that uses a base class should continue to function properly if a subclass of the base class is passed to the component instead

- **Dependency inversion principle**

- Depend on abstractions (i.e., interfaces); do not depend on concretions
- The more a component depends on other concrete components (rather than on the interfaces) the more difficult it will be to extend

- **Interface segregation principle**

- Many client-specific interfaces are better than one general purpose interface
- For a server class, specialized interfaces should be created to serve major categories of clients
- Only those operations that are relevant to a particular category of clients should be specified in the interface

- **Component Packaging Principles**

- **Release reuse equivalency principle**

- The granularity of reuse is the granularity of release
- Group the reusable classes into packages that can be managed, upgraded, and controlled as newer versions are created

- **Common closure principle**

- Classes that change together belong together
- Classes should be packaged cohesively; they should address the same functional or behavioral area on the assumption that if one class experiences a change then they all will experience a change

- **Common reuse principle**

- Classes that aren't reused together should not be grouped together
- Classes that are grouped together may go through unnecessary integration and testing when they have experienced no changes but when other classes in the package have been upgraded

- **Components**
- Establish naming conventions for components that are specified as part of the architectural model and then refined and elaborated as part of the component level model
- Obtain architectural component names from the problem domain and ensure that they have meaning to all stakeholders who view the architectural model (e.g., Calculator)
- Use infrastructure component names that reflect their implementation specific meaning (e.g., Stack)

- Dependencies and inheritance in UML
- Model any dependencies from left to right and inheritance from top (base class) to bottom (derived classes)
- Consider modeling any component dependencies as interfaces rather than representing them as a direct component-to-component dependency

- **Cohesion**

- Cohesion is the "single-mindedness' of a component
- It implies that a component or class encapsulates only attributes and operations that are closely related to one another and to the class or component itself

- The objective is to keep cohesion as high as possible

- The kinds of cohesion can be ranked in order from highest (best) to lowest (worst)

- **Functional**

- A module performs one and only one computation and then returns a result

- **Layer**

- A higher layer component accesses the services of a lower layer component

- **Communicational**

- All operations that access the same data are defined within one class

- **Kinds of cohesion**

- **Sequential**

- Components or operations are grouped in a manner that allows the first to provide input to the next and so on in order to implement a sequence of operations

- **Procedural**

- Components or operations are grouped in a manner that allows one to be invoked immediately after the preceding one was invoked, even when no data passed between them

- Temporal Operations are grouped to perform a specific behavior or establish a certain state such as program start-up or when an error is detected

- **Utility**

- Components, classes, or operations are grouped within the same category because of similar general functions but are otherwise unrelated to each other

Coupling

- As the amount of communication and collaboration increases between operations and classes, the complexity of the computer-based system also increases
- As complexity rises, the difficulty of implementing, testing, and maintaining software also increases
- Coupling is a qualitative measure of the degree to which operations and classes are connected to one another
- The objective is to keep coupling as low as possible
- The kinds of coupling can be ranked in order from lowest (best) to highest (worst)
- **Data coupling**
- Operation A() passes one or more atomic data operands to operation B(); the less the number of operands, the lower the level of coupling
- **Stamp coupling**
- A whole data structure or class instantiation is passed as a parameter to an operation – Control coupling
- Operation A() invokes operation B() and passes a control flag to B that directs logical flow within B()
- Consequently, a change in B() can require a change to be made to the meaning of the control flag passed by A(), otherwise an error may result
- **Common coupling**
- A number of components all make use of a global variable, which can lead to uncontrolled error propagation and unforeseen side effects

- **Content coupling**
- One component secretly modifies data that is stored internally in another component

Conducting Component-Level Design

1) Identify all design classes that correspond to the problem domain as defined in the analysis model and architectural model

2) Identify all design classes that correspond to the infrastructure domain

These classes are usually not present in the analysis or architectural models

- These classes include GUI components, operating system components, data
- management components, networking components, etc.

3) Elaborate all design classes that are not acquired as reusable components

a) Specify message details (i.e., structure) when classes or components collaborate

b) Identify appropriate interfaces (e.g., abstract classes) for each component

c) Elaborate attributes and define data types and data structures required to implement them (usually in the planned implementation language) d) Describe processing flow within each operation in detail by means of pseudo code or UML activity diagrams

4) Describe persistent data sources (databases and files) and identify the classes required to manage them

5) Develop and elaborate behavioral representations for a class or component

This can be done by elaborating the UML state diagrams created for the analysis model and by examining all use cases that are relevant to the design class

6) Elaborate deployment diagrams to provide additional implementation detail

- Illustrate the location of key packages or classes of components in a system by using class instances and designating specific hardware and operating system environments

7) Factor every component-level design representation and always consider alternatives

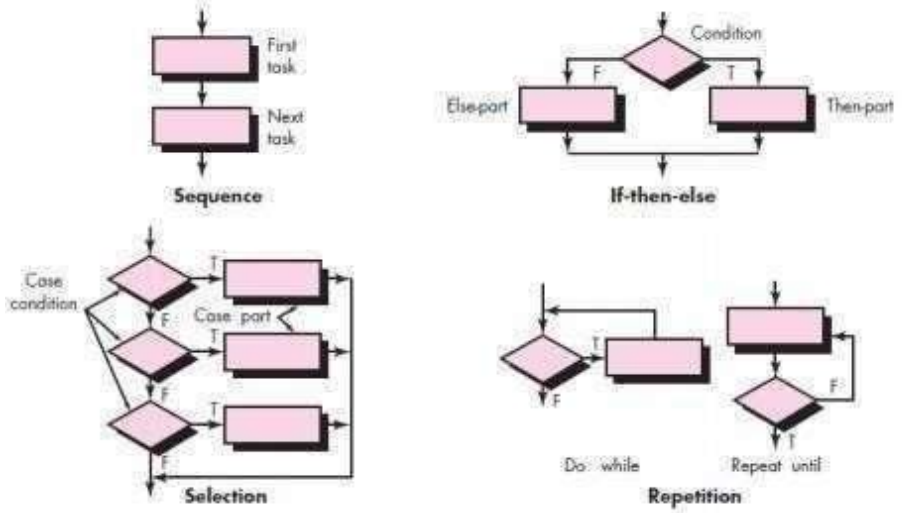
- Experienced designers consider all (or most) of the alternative design solutions before settling on the final design model
- The final decision can be made by using established design principles and guidelines

Designing traditional Components

- Conventional design constructs emphasize the maintainability of a functional/procedural program
- Sequence, condition, and repetition
- Each construct has a predictable logical structure where control enters at the top and exits at the bottom, enabling a maintainer to easily follow the procedural flow
- **Various notations depict the use of these constructs**
- Graphical design notation
- Sequence, if-then-else, selection, repetition (see next slide)
- Tabular design notation (see upcoming slide)
- Program design language
- Similar to a programming language; however, it uses narrative text embedded directly within the program statements

Graphical Design Notation

- Graphical tools, such as the UML activity diagram or the flowchart, provide useful pictorial patterns that readily depict procedural detail.
- The activity diagram allows you to represent sequence, condition, and repetition - all elements of structured programming and is a descendent of an earlier pictorial design representation (still used widely) called a flowchart



GROUP OF INSTITUTIONS

- A box is used to indicate a processing step.
- A diamond represents a logical condition, and arrows show the flow of control
- The sequence is represented as two processing boxes connected by a line (arrow) of control.
- Condition, also called if-then-else, is depicted as a decision diamond that, if true, causes then-part processing to occur, and if false, invokes else-part processing.
- Repetition is represented using two slightly different forms.
- The do while tests a condition and executes a loop task repetitively as long as the condition holds true.

- A repeat until executes the loop task first and then testsa condition and repeats the
- task until the condition fails.
- The selection (or select-case) is actually an extension of the if-then-else.
- A parameter is tested by successive decisions until a true condition occurs and a case part processing path is executed.
- Tabular Design Notation
- List all actions that can be associated with a specific procedure (or module)
- List all conditions (or decisions made) during execution of the procedure
- Associate specific sets of conditions with specific actions, eliminating impossible combinations of conditions; alternatively, develop every possible permutation of conditions
- Define rules by indicating what action(s) occurs for a set of conditions

	Rules								
Conditions	**1**	**2**	**3**	**4**					n
Condition #1	✓			✓	✓				
Condition #2		✓		✓					
Condition #3			✓		✓				
Actions									
Action #1	✓			✓	✓				
Action #2		✓		✓					
Action #3			✓						
Action #4			✓	✓	✓				
Action #5	✓	✓			✓				

- **Program Design Language**
- Program design language (PDL), also called structured English or pseudo code, incorporates the logical structure of a programming language with the free-form expressive ability of a natural language (e.g., English).
- Narrative text (e.g., English) is embedded within a programming language-like syntax. Automated tools can be used to enhance the application of PDL
- A basic PDL syntax should include constructs for component definition, interface description, data declaration, block structuring, condition constructs, repetition constructs, and input-output (I/O) constructs.
- It should be noted that PDL can be extended to include keywords for multitasking and/or concurrent processing, interrupt handling, interprocess synchronization, and many other features.
- The application design for which PDL is to be used should dictate the final form for the design language

R.M.K GROUP OF INSTITUTIONS

USER INTERFACE DESIGN:

If a product is to be successful, it must exhibit good usability—a qualitative measure of the ease and efficiency with which a human can employ the functions and features offered by the high-technology product

Interface Design:

Interface design focuses on the following :

The design of interfaces between software components

The design of interfaces between the software and other nonhuman producers and consumers of information

The design of the interface between a human and the computer

R.M.K GROUP OF INSTITUTIONS

Golden Rules

- Place the user in control
- Reduce the user's memory load
- Make the interface consistent

Place the User in Control

- Define interaction modes in a way that does not force a user into unnecessary or undesired actions
- The user shall be able to enter and exit a mode with little or no effort (e.g., spell check edit text spell check)
- Provide for flexible interaction – The user shall be able to perform the same action via keyboard commands, mouse movement, or voice recognition
- Allow user interaction to be interruptible and "undo" able – The user shall be able to easily interrupt a sequence of actions to do something else (without losing the work that has been done so far) – The user shall be able to "undo" any action
- Streamline interaction as skill levels advance and allow the interaction to be customized – The user shall be able to use a macro mechanism to perform a sequence of repeated interactions and to customize the interface
- Hide technical internals from the casual user – The user shall not be required to directly use operating system, file management, networking. etc., commands to perform any actions. Instead, these operations shall be hidden from the user and performed "behind the scenes" in the form of a real-world abstraction
- Design for direct interaction with objects that appear on the screen The user shall be able to manipulate objects on the screen in a manner similar to what would occur if the object were a physical thing (e.g., stretch a rectangle, press a button, move a slider)

- **Reduce the User's Memory Load**
- Reduce demand on short-term memory – The interface shall reduce the user's requirement to remember past actions and results by providing visual cues of such actions
- Establish meaningful defaults – The system shall provide the user with default values that make sense to the average user but allow the user to change these defaults – The user shall be able to easily reset any value to its original default value
- Define shortcuts that are intuitive The user shall be provided mnemonics (i.e., control or alt combinations) that tie easily to the action in a way that is easy to remember such as the first letter The visual layout of the interface should be based on a real world metaphor – The screen layout of the user interface shall contain well-understood visual cues that the user can relate to real-world actions
- •Disclose information in a progressive fashion – When interacting with a task, an object or some behavior, the interface shall be organized hierarchically by moving the user progressively in a step-wise fashion from an abstract concept to a concrete action (e.g., text format options format dialog box)
- **Make the Interface Consistent**
- The interface should present and acquire information in a consistent fashion – All visual information shall be organized according to a design standard that is maintained throughout all screen displays – Input mechanisms shall be constrained to a limited set that is used consistently throughout the application – Mechanisms for navigating from task to task shall be consistently defined and implemented
- Allow the user to put the current task into a meaningful context – The interface shall provide indicators (e.g., window titles, consistent color coding) that enable the user to know the context of the work at hand

- Maintain consistency across a family of applications – A set of applications performing complimentary functionality shall all implement the same design rules so that consistency is maintained for all interaction

- If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so – Once a particular interactive sequence has become a de facto standard (e.g., alt-S to save a file), the application shall continue this expectation in every part of its functionality

Interface Analysis and Design Process Model

- User interface development follows a spiral process

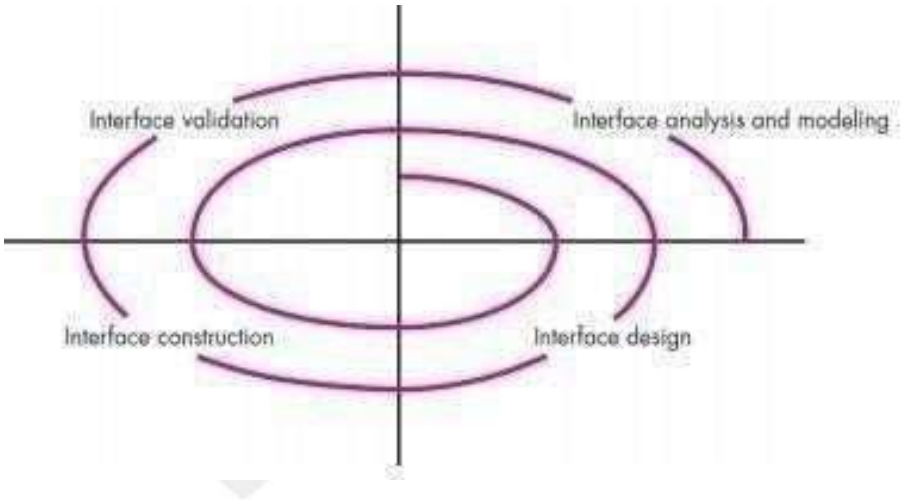


Fig : User Interface design process

- **Interface analysis (user, task, and environment analysis)**
- Focuses on the profile of the users who will interact with the system
- Concentrates on users, tasks, content and work environment
- Studies different models of system function (as perceived from the outside)
- Delineates the human- and computer-oriented tasks that are required to achieve system function

- **Interface design**
- Defines a set of interface objects and actions (and their screen representations) that enable a user to perform all defined tasks in a manner that meets every usability goal defined for the system

- **Interface construction**
- Begins with a prototype that enables usage scenarios to be evaluated
- Continues with development tools to complete the construction
- **Interface validation**
- The ability of the interface to implement every user task correctly, to accommodate all task variations, and to achieve all general user requirements
- The degree to which the interface is easy to use and easy to learn
- The users' acceptance of the interface as a useful tool in their work
- **User Interface Analysis**
- To perform user interface analysis, the practitioner needs to study and understand four elements
- The users who will interact with the system through the interface
- The tasks that end users must perform to do their work
- The content that is presented as part of the interface
- The work environment in which these tasks will be conducted
- **User Analysis**
- The analyst strives to get the end user's mental model and the design model to converge by understanding
- The users themselves

- How these people use the system
- Information can be obtained from
- User interviews with the end users
- Sales input from the sales people who interact with customers and users on a regular basis
- Marketing input based on a market analysis to understand how different population segments might use the software
- Support input from the support staff who are aware of what works and what doesn't, what users like and dislike, what features generate questions, and what features are easy to use
- A set of questions should be answered during user analysis
- **Task Analysis and Modeling**
- Task analysis strives to know and understand
- The work the user performs in specific circumstances
- The tasks and subtasks that will be performed as the user does the work
- The specific problem domain objects that the user manipulates as work is performed
- The sequence of work tasks (i.e., the workflow)
- The hierarchy of tasks
- Use cases – Show how an end user performs some specific work-related task
- Enable the software engineer to extract tasks, objects, and overall workflow of the interaction
- Helps the software engineer to identify additional helpful features

- Task elaboration refines interactive tasks
- Object elaboration identifies interface objects (classes)
- Workflow analysis defines how a work process is completed when several people (and roles) are involved
- **Analysis of Display Content**
- For modern applications, display content can range from character-based reports (e.g., a spreadsheet), graphical displays (e.g., a histogram, a 3-D model a picture of a person), or specialized information (e.g., audio or video files).
- During this interface analysis step, the format and aesthetics of the content (as it is displayed by the interface) are considered.
- Among the questions that are asked and answered are:
- Are various types of data assigned to consistent locations on the screen (e.g., photos always in upper right corner)?
- Are users able to customize the screen location for content?
- Is proper on-screen identification assigned to all content?
- Can large reports be partitioned for ease of understanding?
- Are mechanisms available for moving directly to summary information for large collections of data?
- Is graphical output scaled to fit within the bounds of the display device that is used?
- How is color used to enhance understanding?
- How are error messages and warnings presented in order to make them quick and easy to see and understand?
- The answers to these (and other) questions will help you to establish requirements for content presentation

- Analysis of the Work Environment
- Software products need to be designed to fit into the work environment, otherwise they may be difficult or frustrating to use

- Factors to consider include
- Type of lighting, Display size and height, Keyboard size, height and ease of use, Mouse type and ease of use ,Surrounding noise, Space limitations for computer and/or user ,Weather or other atmospheric conditions, Temperature or pressure restrictions ,Time restrictions (when, how fast, and for how long)

- Interface Design Steps
- Using information developed during interface analysis, define interface objects and actions (operations).

- Define events (user actions) that will cause the state of the user interface to change. Model this behavior.

- Depict each interface state as it will actually look to the end-user.
- Indicate how the user interprets the state of the system from information provided through the interface.

- **Design Issues**
- Response time
- Help facilities
- Error handling
- Menu and command labeling
- Application accessibility

- Internationalization

PATTERN-BASED DESIGN

Each of us has encountered a design problem and silently thought: I wonder if anyone has developed a solution to for this?

> What if there was a standard way of describing a problem (so you could look it up), and an organized method for representing the solution to the problem?

Design patterns are a codified method for describing problems and their solution allows the software engineering community to capture design knowledge in a way that enables it to be reused.

DESIGN PATTERNS

A design pattern can be characterized as "a three-part rule which expresses a relation between a certain context, a problem, and a solution" [Ale79]. For software design, context allows the reader to understand the environment in which the problem resides and what solution might be appropriate within that environment. A set of requirements, including limitations and constraints, acts as a system of forces that influences how the problem can be interpreted within its context and how the solution can be effectively applied. It is reasonable to argue that most problems have multiple solutions, but that a solution is effective only if it is appropriate within the context of the existing problem. It is the system of forces that causes a designer to choose a specific solution. The intent is to provide a solution that best satisfies the system of forces, even when these forces are contradictory. Finally, every solution has consequences that may have an impact on other aspects of the software and may themselves become part of the system of forces for other problems to be solved within the larger system.

R.M.K GROUP OF INSTITUTIONS

Coplien [Cop05] characterizes an effective design pattern in the following way:

- It solves a problem: Patterns capture solutions, not just abstract principles or strategies.
- It is a proven concept: Patterns capture solutions with a track record, not theories or speculation.
- The solution isn't obvious: Many problem-solving techniques (such as software design paradigms or methods) try to derive solutions from first principles. The best patterns generate a solution to a problem indirectly—a necessary approach for the most diffi cult problems of design.
- It describes a relationship: Patterns don't just describe modules, but describe deeper system structures and mechanisms.
- The pattern has a significant human component (minimize human intervention). All software serves human comfort or quality of life; the best patterns explicitly appeal to aesthetics and utility. A design pattern saves you from "reinventing the wheel," or worse, inventing a "new wheel" that is slightly out of round, too small for its intended use, and too narrow for the ground it will roll over. Design patterns, if used effectively, will invariably make you a better software designer.

GENERATIVE & NON-GENERATOVE PATTERNS:

Non-Generative patterns describe a context and a problem but it does not provide any clear-cut solution. Eg-RubberNecking

Generative patterns describe an important and repeatable aspect of a system and then provide us with a way to build that aspect within a system of forces that are unique to a given context.
A collection of generative design patterns could be used to "generate" an application or computer-based system whose architecture enables it to adapt to change.

KINDS OF PATTERNS:

Architectural patterns describe broad-based design problems that are solved using a structural approach.

Data patterns describe recurring data-oriented problems and the data modeling solutions that can be used to solve them.

Component patterns (also referred to as design patterns) address problems associated with the development of subsystems and components, the manner in which they communicate with one another, and their placement within a larger architecture

Interface design patterns describe common user interface problems and their solution with a system of forces that includes the specific characteristics of end-users.

WebApp patterns address a problem set that is encountered when building WebApps and often incorporates many of the other patterns categories just mentioned.

KINDS OF PATTERNS IN OO DESIGN

Creationalpatternsfocus on the "creation, composition, and representation of objects

Structuralpatternsfocus on problems and solutions associated with how classes and objects are organized and integrated to build a larger structure

Behavioralpatternsaddress problems associated with the assignment of responsibility between objects and the manner in which communication is effected between objects

R.M.K GROUP OF INSTITUTIONS

PATTERN BASED SOFTWARE DESIGN

The best designers in any field have an uncanny ability to see patterns that characterize a problem and corresponding patterns that can be combined to create a solution. Throughout the design process, you should look for every opportunity to apply existing design patterns (when they meet the needs of the design) rather than creating new ones.

Pattern-Based Design in Context

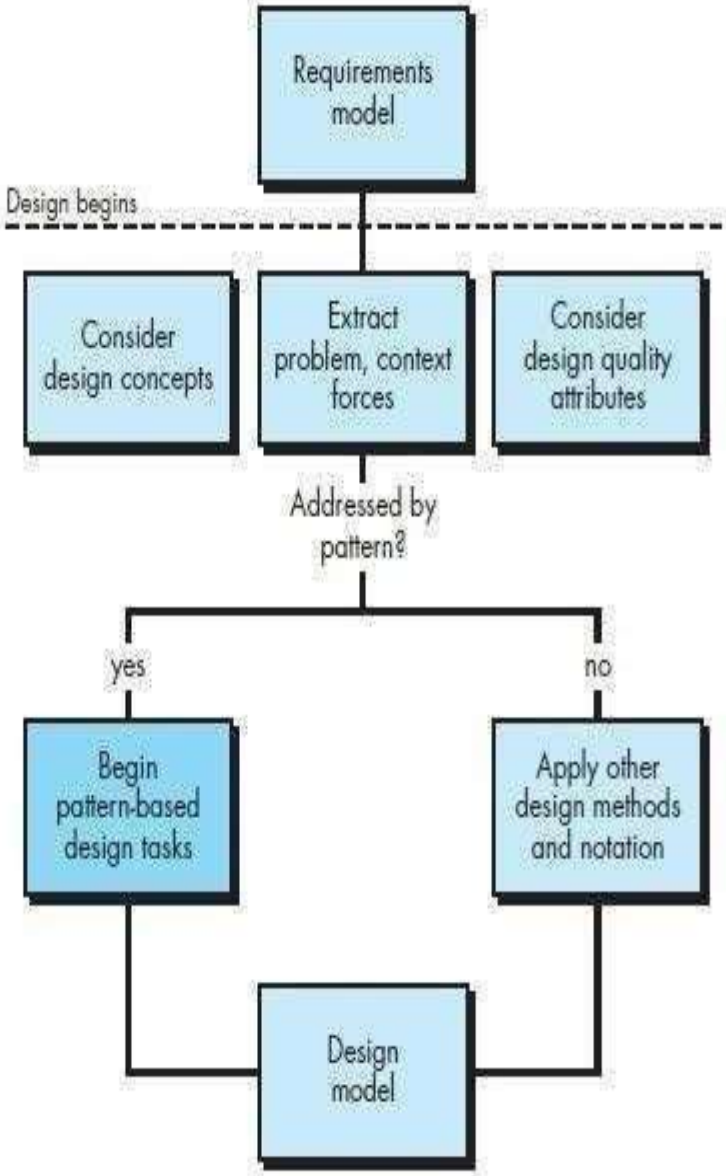
Pattern-based design is not used in a vacuum. The concepts and techniques discussed for architectural, component-level, and user interface design are all used in conjunction with a pattern-based approach.

R.M.K GROUP OF INSTITUTIONS

To accomplish this, you'll use the methods and modeling tools available for architectural, component-level, and interface design. But only when you're faced with a problem, context, and system of forces that have not been solved before. If a solution already exists, use it! And that means applying a pattern-based design approach.

FIGURE 16.1

Pattern-based design in context



R.M.K GROUP OF INSTITUTIONS

Thinking in Patterns

Pattern-based design implies with a "new way of thinking" [Sha05l that begins by considering context—the big picture. As context is evaluated, you extract a hier- archy of problems that must be solved. Some of those problems will be global in nature, while others will address specific features and functions of the software. All will be affected by a system of forces that will influence the nature of the solution that is proposed.

Shallo way and Trott [Sha05l suggest the following approach* that enables a designer to think in patterns:

1. Be sure you understand the big picture—the context in which the software to be built resides. The requirements model should communicate this to you.
2. Examining the big picture, extract the patterns that are present at that level of abstraction.
3. Begin your design with "big picture" patterns that establish a context or skeleton for further design work.
4. "Work inward from the context' look.ing for patterns at lower levels of abstraction that contribute to the design solution.
5. Repeat steps 1 to 4 until the complete design is fleshed out.
6. Refine the design by adapting each pattern to the specifics of the software you're trying to build. A simplified pattern template is shown

INFO

Design Pattern Template

Pattern name—describes the essence of the pattern in a short but expressive name

Problem—describes the problem that the pattern addresses

Motivation—provides an example of the problem

Context—describes the environment in which the problem resides including the application domain

Forces—lists the system of forces that affect the manner in which the problem must be solved; includes a discussion of limitation and constraints that must be considered

Solution—provides a detailed description of the solution proposed for the problem

Intent—describes the pattern and what it does

Collaborations—describes how other patterns contribute to the solution

Consequences—describes the potential trade-offs that must be considered when the pattern is implemented and the consequences of using the pattern

Implementation—identifies special issues that should be considered when implementing the pattern

Known uses—provides examples of actual uses of the design pattern in real applications

Related patterns—cross-references related design patterns

R.M.K GROUP OF INSTITUTIONS

It's important to note that patterns are not independent entities. Design pat- terns that are present at a high level of abstraction will invariably influence the manner in which other patterns are applied at lower levels of abstraction. In addition, patterns often collaborate with one another. The implication when you select an architectural pattern, it may very well influence the component-level design patterns you choose. Likewise, when you select a specific interface design pattern, you are sometimes forced to use other patterns that collaborate with it.

To illustrate, consider the SofeHomeAssured.com Web App. If you consider the big picture, the Web App must address how to provide information about Sofe Home products and services, how to sell SafeHome products and services to customers, and how to establish Internet-based monitoring and control of an installed security system. Each of these fundamental problems can be further refined into a set of sub problems.

For example, How to sell via the Internet implies an E-commerce pattern that itself implies a large number of patterns at lower levels of abstraction. The E-commerce pattern likely, an architectural pattern) implies mechanisms for setting up a customer account, displaying the products to be sold, selecting products for purchase, and so forth. Hence, if you think in patterns, it is important to determine whether a pattern for setting up an account exists. If Set Up Account is available as a viable pattern for the problem context, it may collaborate with other patterns such as BuildInputForm, Manage Forms Input, and Validate Forms- Entry. Each of these patterns delineates problems to be solved and solutions that may be applied.

R.M.K GROUP OF INSTITUTIONS

Design Tasks

The following design tasks are applied when a pattern-based design philosophy is used

1. Examine the requirements model and develop a problem hierarchy. Describe each problem and sub problem by isolating the problem, the context, and the system of forces that apply. Work from broad problems (high level of abstraction) to smaller sub problems (at lower levels of abstraction).

2. Determine if a reliable pattern language has been developed for the problem domain.

The Safe Home software team would look for a pattern language developed specifically for home security products. If that level of pattern language specificity could not be found, the team would partition the Safe Home software problem into a series of generic problem domains digital device monitoring problems, user interface problems.Digital video management problems and search for appropriate pat- tern languages.

3. Beginning with a broad problem, determine whether one or more architectural patterns are available for it. If an architectural pattern is avail- able, be certain to examine all collaborating patterns. If the pattern is appropriate, adapt the design solution proposed and build a design model element that adequately represents it. For example, a broad problem for the SafeHomeAssured.com WebApp is addressed with an E-commerce pattern This pattern will suggest a specific architecture for addressing e-commerce requirements.

4. Using the collaborations provided for the architectural pattern, examine subsystem or component-level problems and search for appropriate patterns to address them. It may be necessary to search through other pattern repositories as well as the list of patterns that corresponds to the architectural solution. If an appropriate pattern is found, adapt the design solution proposed and build a design model element that adequately rep- resents it.

R.M.K GROUP OF INSTITUTIONS

Be certain to apply step 7.

5. Repeat steps 2 through 4 until all broad problems have been addressed. The implication is to begin with the big picture and elaborate to solve problems at increasingly more detailed levels.

6. If user interface design problems have been isolated (this is almost always the easel, search the many user interface design pattern repositories for appropriate patterns. Proceed in a manner similar to steps 3, 4, and 5.

7. Regardless of its level of abstraction, if a pattern language andZor patterns repository or individual pattern shows promise, compare the problem to be solved against the existing patterns presented.

8. Be certain to examine context and forces to ensure that the pattern does, in fact, provide a solution that is amenable to the problem .Be certain to refine the design as it is derived from patterns using design quality criteria as a guide.

Building a Pattern-Organizing Table

As pattern-based design proceeds, you may encounter trouble organizing and categorizing candidate patterns from multiple pattern languages and repositories. To help organize your evaluation of candidate patterns, Microsoft [Micl3l suggests the creation of a pattern organizing table that takes the general form shown in Figure 16.2.

A pattern-organizing table can be implemented as a spreadsheet model using

the form shown in the figure. An abbreviated list of problem statements, organized by data/content, architecture, component level, and user interface issues, is presented in the left-hand column. Four pattern types—database, application, implementation, and infrastructure are listed across the top row. The names of candidate patterns are noted in the cells of the table.

To provide entries for the organizing table, you'll search through pattern languages and repositories for patterns that address a particular problem statement. When one or more candidate patterns is found, it is entered in the row corresponding to the problem statement and the column that corresponds to the pattern type. The name of the pattern is entered as a hyperlink to the URL of the Web address that contains a complete description of the pattern.

R.M.K GROUP OF INSTITUTIONS

A Pattern Organizing Table

FIGURE 16.2

A pattern-organizing table

Source: Adapted from [Mic04].

	Database	Application	Implementation	Infrastructure
Data/Content				
Problem statement ...	PatternName(s)		PatternName(s)	
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...	PatternName(s)			PatternName(s)
Architecture				
Problem statement ...		PatternName(s)		
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...				
Component-level				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...				PatternName(s)
Problem statement ...		PatternName(s)	PatternName(s)	
User interface				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	

Assignment with co and K level

Suggest a design Pattern that you encounter in a category of everyday things (e.g. consumer electronics, autmobiles and appliances)(CO3,K2)

R.M.K
GROUP OF
INSTITUTIONS

PART A (Q& A) (with K level and CO)

1. **Define the term: Abstraction**. (CO4,K2)

- The psychological notation of abstraction permits one to concentrate on a problem at some level of generalization without regard to irrelevant low level details; use of abstraction also permits one to work with concepts and terms that are familiar in the problem environment without having to transform tem to an unfamiliar structure.

2) **In what way abstraction differs from refinement.** CO4,K3)

- Abstraction and refinement are complementary steps.
- The major difference is that – in the abstraction low – level details are suppressed.
- Refinement helps the designer to elaborate low – level details.

3) **Name three levels of abstraction (**CO4,K3)

- Procedural Abstraction
- Data Abstraction
- Control Abstraction

4) **What are the criteria for the effective modular design?** . (CO4,K2)

- Modular Decomposability
- Modular Compatibility
- Modular Understandability
- Modular continuity
- Modular Protection

5. **What is mean by user interface design?** (CO4,K2)

- User Interface Design creates an effective communication medium between a human and a computer

6. **Define the term Software Architecture.** (CO4,K2)

- It is the hierarchal structure of software components and their interactions. In software architecture the software model is designed. The structure of that model is partitioned horizontally or vertically.

7) **What the advantages of modular design?** (CO4,K2)

- Reduces complexity
- Facilitates change
- Easier implementation

8) **What are the quality parameters considered for effective modular design?** (CO4,K2)

- Function independence
- Cohesion
- Coupling

9) **What are types of cohesion?** (CO4,K2)

- Coincidentally Cohesion
- Logically Cohesion
- Temporal Cohesion
- Procedural Cohesion
- Communication Cohesion
-

10. **What are types of coupling?** (CO4,K2)

- Data Coupling
- Stamp coupling
- Control Coupling
- External coupling
- Common coupling
-

Content Coupling

11. Mention the types of design models.
(CO4,K2) Data design

Architectural design

Interface design

Component – level design

12. Why the Architecture is Important? (CO4,K2)

Representations of software architecture are an enabler for communication between all parties interested in the development of computer based system

The architecture highlights early design decisions that will have profound impact on all software engineering work that follows and, as important, on the ultimate success of the system as an operational entity.

Architecture "constitutes a relatively small, intellectually graspable model of how the system is structured and how its components work together".

13. Mention the activities of user interface design process. (CO4,K2)

User task and Environmental analysis and modeling Interface Design

Interface Construction

Interface Validation

14. What is Pattern Based Design ? (CO4,K2)

Pattern based design is not used in a vaccum.The concepts and techniques discussed for architectural ,component level and user interface design are all used in conjuction with a pattern based approach

15. Give any two Design task.

1. Examine the requirements model and develop a problem hierarchy
2. Determine if a reliable pattern language has been developed for the problem domain.

PART B –UNIT III(with K level and CO)

Explain in detail about Design Process(CO4,K1)

Explain in detail about Design concepts (CO4,K3)

Explain in detail about Design Model (CO4,K2)

Explain in detail about Architectural design(CO4,K3)

Explain in detail about Component Level Design

What is traditional components ? Explain(CO4,K2)

What is Pattern-Based Software Design(CO4,K1)

what is Designing Traditional Components?Explain (CO4,K1)

Explain the core activities involved in User Interface design process with necessary block diagram (CO4,K3)

Explain the basic concepts of software design (CO4,K3)

Explain clearly the concept of coupling & cohesion? For each type of coupling give an example of two components coupled in that way? (CO4,K3)

What is software architecture ? Describe in detail different types of software architectural styles with illustrations. (CO4,K3)

SUPPORTIVE ONLINE COURSES - UNIT III

- https://www.coursera.org/specializations/software-design-architecture
- https://www.coursera.org/learn/software-design-methods-tools
- https://www.coursera.org/learn/software-design-development-life-cycle

R.M.K GROUP OF INSTITUTIONS

Supportive online Certification courses

NPTEL/OTHER REFERENCES / WEBSITES:

http://nptel.ac.in
Software Engineering Tutorial – javatpoint

https://www.tutorialspoint.com/software_engineerin

g/

SoftwareEngineeringTutorial for Beginners:Learn in 3 Days (guru99.com)

Software Engineering Tutorial | A CompleteGuide for Beginners

(educba.com)

www.udemy.com

www.upgrad.com

www.simplilearn.com

Real time Applications in day to day life and to Industry

1. To explain the concept of a real-time system and why these systems are usually implemented as concurrent processes

2. To describe a design process for real-time systems

3. To explain the role of a real-time operating system

4. To introduce generic process architectures for monitoring and control and data acquisition systems

R.M.K GROUP OF INSTITUTIONS

CONTENTS BEYOND THE SYLLABUS UI DESIGN TOOLS

UI design tools give designers what they need to design accurate hi-fi wireframes, mockups, and prototypes and render minimally viable products. They represent the nuts and bolts of a design, communicating its functionality.

UX design tools focus on the user and how they'll experience the content. These tools can help structure the information architecture, as well as how someone will flow through the experience. We're going to take a look at some UI and UX tools that you may find handy for your design process. Some of these tools have features that work in both UX Or UI, but for the sake of your own user experience, we've divided the list into two categories With a full suite of applications, In vision gives designers all of the UI design tools they need to create fully realized and functional prototypes with dynamic elements and animations.

Along with these easy-to-use UI design tools, they also make communication easy — with collaboration features that let developers share their work as they design it, receive feedback, and make documented changes at each step. Another useful aspect of In Vision is the digital whiteboard that allows team members to get their ideas out there, interact, and get that all important sign-off before moving forward.

Self-Learning/Enrichment Topics

Axure functions in prototyping and keeping track of the workflow. It features a smooth interface to document as you go. High fidelity drives this app, resulting in prototypes full of details.

Axure offers many of the other features of popular prototyping and UI design tools. It allows for testing of functionality and puts everything together for an easy developer handoff. This, combined with an emphasis on communication, ensures that everyone on a project is kept up-to-date with progress and changes as they happen in real time, making Axure a solid choice for UI design.

Craft a plugin from In Vision, works right alongside what you might be doing in Photoshop or Sketch, with a sync function that updates what you're working on. Along with this time-saving feature, Craft offers everything you need for prototyping and collaboration. Craft sets itself apart from other UI design tools with its placeholder content. You get access to both Getty and iStock photos, letting you fill your layout with better visuals. And if there's data in your layout, you can use your own or bring it in from other sources. Not many UI design tools let you fill your mockups with more meaningful content.

Framer Xhas evolved from its early days as a code-only prototyping application. Today it offers a host of UI design tools for building functional prototypes and testing for usability. Its ability to work with react makes it ideal for

UI designers who like to stay on top of the newest web design innovations.

Framer X also offers a variety of plugins in their store, giving UI designers features such as UI kits for integrating social media channels like Snap chat and Twitter, players for embedding a variety of media, grids, and other useful components that can be easily integrated. It's a great interface design tool with a fairly east learning curve.

16.Assessment Schedule (Proposed Date & Actual Date)

Sl. No.	ASSESSMENT	Proposed Date	Actual Date
1	FIRST INTERNAL ASSESSMENT		
2	SECOND INTERNAL ASSESSMENT		
3	MODEL EXAMINATION		
4	END SEMESTER EXAMINATION		

R.M.K GROUP OF INSTITUTIONS

PRESCRIBED TEXTBOOKS & REFERENCE BOOKS

TEXT BOOKS:

1. Roger S. Pressman, "Software Engineering – A Practitioner's Approach", Seventh Edition, Mc Graw-Hill International Edition, 2010.

REFERENCES:

1. Ian Sommerville, "Software Engineering", Tenth Edition, Pearson Education, 2016.
2. Ivar Jacobson, Harold Bud Lawson, Pan-Wei Ng, Paul E. McMahon, Michael Goedicke, "The Essentials of Modern Software Engineering", Morgan & Claypool Publishers, 2019.
3. Rajib Mall, "Fundamentals of Software Engineering", PHI Learning Private Limited, Fourth Edition,2014.
4. Karl Wiegers,"Software Requirements - Best Practices", Microsoft Press US, 3rd Edition, 2013.
5. David P. Voorhees, "Guide to Efficient Software Design: An MVC Approach to Concepts, Structures, and Models", Springer; 1st ed, 2020.
6. Gerard O'Regan, "Concise Guide to Software Testing", 1st ed, Springer, 2019.
7. Duane Petersen, "Transforming Project Management: An Essential Paradigm for Turning Your Strategic Planning into Action", McGraw-Hill Education, 1st edition, 2021.

Mini Project / Activity

- Designing a User Interface for Web Based Project Management (CO3-K5)
- Medical Information System (CO3-K5)
- Architectural Design- Students Management Systems (CO3-K6)

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.